

```

// === БЕЗОПАСНАЯ ЗАГРУЗКА ЛЮБЫХ КАРТИНОК ЧЕРЕЗ WIC НА ВИДЕОКАРТУ ===

HRESULT LoadTextureFromFile(const WCHAR* szFileName)
{
    HRESULT hr = S_OK;

    // 1. Создаем фабрику декодеров WIC
    IWICImagingFactory* pWICFactory = NULL;

    hr = CoCreateInstance(CLSID_WICImagingFactory, NULL, CLSCTX_INPROC_SERVER,
        __uuidof(IWICImagingFactory), (LPVOID*)&pWICFactory);
    if (FAILED(hr)) return hr;

    // 2. Открываем файл картинки на диске
    IWICBitmapDecoder* pDecoder = NULL;

    hr = pWICFactory->CreateDecoderFromFilename(szFileName, NULL, GENERIC_READ,
        WICDecodeMetadataCacheOnDemand, &pDecoder);
    if (FAILED(hr)) { pWICFactory->Release(); return hr; }

    // 3. Берем самый первый кадр из файла
    IWICBitmapFrameDecode* pFrame = NULL;

    hr = pDecoder->GetFrame(0, &pFrame);
    if (FAILED(hr)) { pDecoder->Release(); pWICFactory->Release(); return hr; }

    // 4. Принудительно конвертируем пиксели в формат RGBA 32-бит
    IWICFormatConverter* pConverter = NULL;

    hr = pWICFactory->CreateFormatConverter(&pConverter);
    if (FAILED(hr)) { pFrame->Release(); pDecoder->Release(); pWICFactory->Release(); return hr; }

    hr = pConverter->Initialize(pFrame, GUID_WICPixelFormat32bppPBGRA, WICBitmapDitherTypeNone,
        NULL, 0.0f, WICBitmapPaletteTypeCustom);
    if (FAILED(hr)) { pConverter->Release(); pFrame->Release(); pDecoder->Release(); pWICFactory-
        >Release(); return hr; }

    // 5. Узнаем физические размеры загруженной картинки
    UINT imgWidth = 0, imgHeight = 0;

```

```

pConverter->GetSize(&imgWidth, &imgHeight);

if (pPixelsBuffer != NULL) // Если память под кадр уже выделена
{
    delete[] pPixelsBuffer; // Физически возвращаем операционной системе старый блок
    pPixelsBuffer = NULL; // Сразу зануляем ярлык, чтобы никто не полез в пустую память
}

if (pCompressedBuffer8x != NULL) // Если память под кадр 8x уже выделена
{
    delete[] pCompressedBuffer8x; // Физически возвращаем операционной системе старый блок
    pCompressedBuffer8x = NULL; // Сразу зануляем ярлык, чтобы никто не полез в пустую
память
}

if (pRawPixels != NULL) // Если память под кадр 8x уже выделена
{
    delete[] pRawPixels; // Физически возвращаем операционной системе старый блок
    pRawPixels = NULL; // Сразу зануляем ярлык, чтобы никто не полез в пустую память
}

g_srcImgWidth = imgWidth;
g_srcImgHeight = imgHeight;

g_pBuffer1_Src = new DWORD[g_srcImgWidth * g_srcImgHeight];

// Вычисляем новые размеры, кратные 8
g_compressedWidth = ((imgWidth + 7) / 8);
g_compressedHeight = ((imgHeight + 7) / 8);

pCompressedBuffer8x = new DWORD[g_compressedWidth * g_compressedHeight];

g_paddedImgWidth = g_compressedWidth * 8;
g_paddedImgHeight = g_compressedHeight * 8;

pPixelsBuffer = new UINT[g_paddedImgWidth * g_paddedImgHeight];

```

```
UINT wicStride = imgWidth * sizeof(UINT);  
hr = pConverter->CopyPixels(NULL, wicStride, wicStride * imgHeight, (BYTE*)g_pBuffer1_Src);  
  
// === ЗОЛОТОЕ ПРАВИЛО: ЧИСТО ЗАКРЫВАЕМ ВСЕ РЕСУРСЫ И ПАМЯТЬ БЕЗ УТЕЧЕК! ===  
pConverter->Release();  
pFrame->Release();  
pDecoder->Release();  
pWICFactory->Release();  
  
return hr;  
}
```